

Arquitetura do Banco de Dados

Event store, outbox, sagas e modelo de leitura

Como o banco de dados do Brado está organizado hoje e as regras que a sua evolução deve seguir: imutabilidade, separação entre escrita e leitura, concorrência, retenção e crescimento.

Brado · Distributed-Event-Messaging-Engine

Arquitetura de dados · versão 1.0 · outubro de 2026

Copyright (©) 2026 Clara Portes

Sumário

01	Princípios	regras que não mudam
02	Visão geral	dois bancos, dois propósitos
03	Convenções	nomes, tipos, tempo e identificadores
04	Lado de escrita: eventos	streams, events, snapshots
05	Lado de escrita: integração	outbox, sagas, inbox
06	Lado de escrita: broker	journal e offsets
07	Lado de leitura	visões desnormalizadas (EF Core)
08	Transações e concorrência	como cada escrita é protegida
09	Imutabilidade e correções	triggers e eventos compensatórios
10	Retenção e manutenção	limpeza, snapshots, backup
11	Evolução futura	o que o banco deve seguir ao crescer
12	Checklist e DDL de referência	implantação e script

Copyright (©) 2026 Clara Portes

01 Princípios

Cinco regras sustentam o modelo de dados. Qualquer mudança futura precisa respeitá-las.

1. O log é a verdade

A tabela `events` é a única fonte de verdade do domínio. Todo o resto — visões, snapshots, sagas — é derivado e pode ser recalculado.

2. Append-only

Eventos nunca são alterados nem apagados. Erros se corrigem com novos eventos. Triggers bloqueiam UPDATE e DELETE.

3. Escrita e leitura separadas

O lado de escrita é normalizado e otimizado para inserção; o de leitura é desnormalizado para consulta e pode estar em outro servidor.

4. Uma transação por decisão

Eventos, versão do stream e mensagens do outbox são gravados juntos. Não existe dual-write entre banco e broker.

5. Idempotência em toda borda

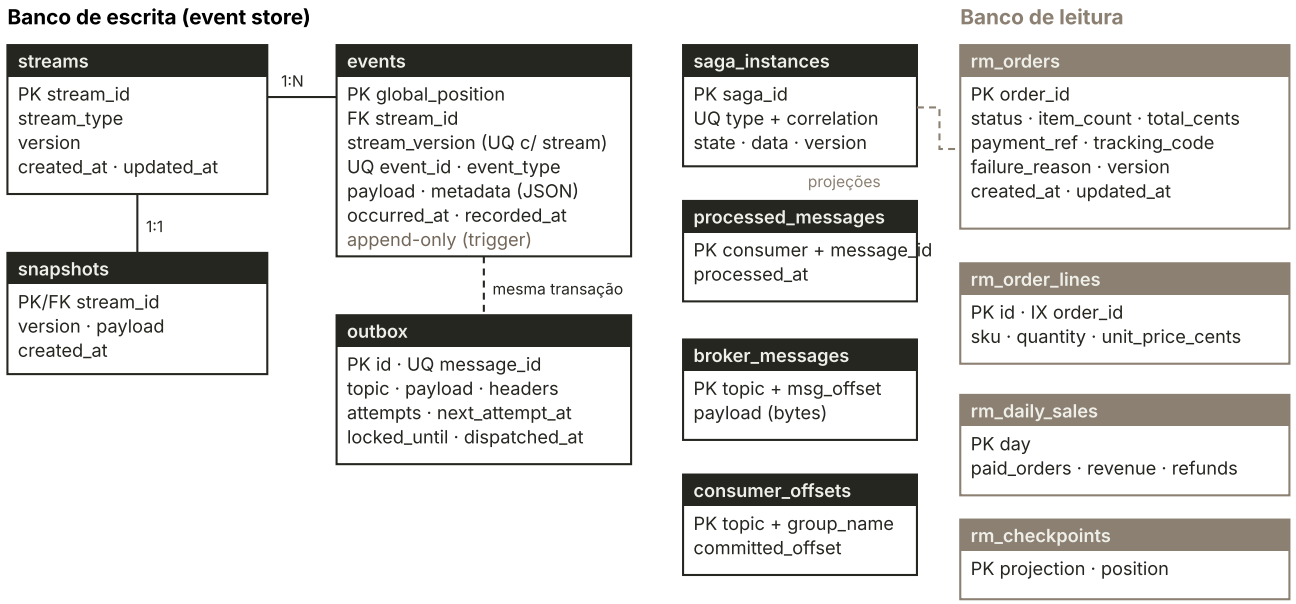
Ids únicos de evento e mensagem, inbox por consumidor e checkpoints transacionais tornam reprocessamentos seguros.

6. Portabilidade controlada

O mesmo modelo roda em PostgreSQL (produção) e SQLite (desenvolvimento). Diferenças ficam isoladas no dialeto.

02 Visão geral

O Brado usa dois bancos lógicos. Eles podem ser o mesmo servidor, mas nunca compartilham tabelas.



Tabelas atuais. Preto: lado de escrita (Dapper + SQL). Vinho: lado de leitura (Entity Framework Core).

Banco	Acesso	Papel	Configuração
Escrita	Dapper + SQL por dialeto	Event store, outbox, sagas, inbox, journal e offsets do broker	`EventStore:Provider` / `ConnectionString`
Leitura	Entity Framework Core	Visões para consulta e checkpoint das projeções	`ReadModel:Provider` / `ConnectionString`

Copyright (©) 2026 Clara Portes

O schema de escrita é criado por um script idempotente (`CREATE ... IF NOT EXISTS`) definido no dialeto. O de leitura é criado pelo EF Core a partir do `ReadModelDbContext` .

03 Convenções

Padrões aplicados a todas as tabelas, atuais e futuras.

Tema	Regra
Nomes	`snake_case` , tabelas no plural; visões de leitura com prefixo `rm_` ; índices `ix_<tabela>_<colunas>` ; únicos `uq_` ; triggers `trg_` .
Tempo	Sempre UTC. PostgreSQL: `TIMESTAMPTZ` . SQLite: texto ISO-8601 com offset `+00:00` , que ordena corretamente como texto.
Identificadores	UUID gerado pela aplicação (v7, ordenável no tempo) gravado como texto para portabilidade. Chaves sequenciais (`global_position` , `id`) só para ordem física.
JSON	PostgreSQL: `JSONB` (inserção via `CAST(@p AS jsonb)`). SQLite: `TEXT` com `CHECK (json_valid(...))` . Nomes de propriedades em camelCase.
Dinheiro	No modelo de leitura em centavos (`BIGINT`) para somar e ordenar em qualquer banco; nos eventos em decimal JSON.
Inteiros	`BIGINT` para versões, offsets e contadores; evita estouro e simplifica o mapeamento.
Streams	`<tipo>-<uuid sem hífens>` , ex.: `order-3f2a91c48b1e44d0...` .

04 Lado de escrita: eventos

Streams, eventos e snapshots formam o núcleo do Event Sourcing.

streams

Uma linha por agregado. Guarda a versão atual e serve como ponto de trava para o controle otimista.

Coluna	PostgreSQL	SQLite	Nulo	Descrição
stream_id	TEXT PK	TEXT PK	não	Identificador do stream.
stream_type	TEXT	TEXT	não	Tipo do agregado (ex.: `order`).
version	BIGINT ≥ 0	INTEGER ≥ 0	não	Quantidade de eventos gravados.
created_at / updated_at	TIMESTAMPTZ	TEXT	não	Criação e último append.

Índice: `ix_streams_type_updated (stream_type, updated_at DESC)` para listar streams recentes por tipo.

events

O log imutável. Cada linha é um fato do domínio com posição global e versão dentro do stream.

Copyright (©) 2026 Clara Portes

Coluna	PostgreSQL	SQLite	Nulo	Descrição
global_position	BIGINT IDENTITY PK	INTEGER AUTOINCREMENT PK	não	Ordem total, usada por projeções e replay.
stream_id	TEXT FK → streams	TEXT FK	não	Stream dono do evento.
stream_version	BIGINT > 0	INTEGER > 0	não	Versão no stream; único junto com `stream_id`.
event_id	TEXT UNIQUE	TEXT UNIQUE	não	UUID v7; também é o `messageId` do outbox.
event_type	TEXT	TEXT	não	Nome estável do evento (ex.: `PaymentProcessed`).
payload	JSONB	TEXT (json_valid)	não	Dados do evento.
metadata	JSONB	TEXT (json_valid)	não	`aggregateType`, `schemaVersion`, `correlationId`.
occurred_at / recorded_at	TIMESTAMPTZ	TEXT	não	Quando aconteceu / quando foi gravado.

Restrições: `uq_events_stream_version (stream_id, stream_version)`, `uq_events_event_id`. Índice: `ix_events_type`. Triggers impedem UPDATE e DELETE.

snapshots

Coluna	PostgreSQL	SQLite	Nulo	Descrição
stream_id	TEXT PK FK	TEXT PK FK	não	Um snapshot (o mais recente) por stream.
version	BIGINT	INTEGER	não	Versão do stream representada.
payload	JSONB	TEXT	não	Estado serializado do agregado.
created_at	TIMESTAMPTZ	TEXT	não	Momento da captura.

Gerado a cada `SnapshotEvery` eventos. Na carga, o repositório restaura o snapshot e aplica apenas os eventos com versão maior. Snapshots são descartáveis: apagá-los só deixa a carga mais lenta.

Copyright (©) 2026 Clara Portes

05 Lado de escrita: integração

Tabelas que conectam o event store ao broker e coordenam processos de longa duração.

outbox

Coluna	PostgreSQL	SQLite	Nulo	Descrição
id	BIGINT IDENTITY PK	INTEGER AUTOINCREMENT PK	não	Ordem de entrega.
message_id	TEXT UNIQUE	TEXT UNIQUE	não	Igual ao `event_id`; chave de deduplicação nos consumidores.
topic	TEXT	TEXT	não	Tópico de destino (ex.: `orders.events`).
payload / headers	JSONB	TEXT	não	Envelope do evento e cabeçalhos.
created_at	TIMESTAMPTZ	TEXT	não	Gravação (mesma transação do evento).
attempts	BIGINT	INTEGER	não	Tentativas que falharam.
next_attempt_at	TIMESTAMPTZ	TEXT	não	Próxima tentativa (backoff).
locked_until / locked_by	TIMESTAMPTZ / TEXT	TEXT	sim	Lease do worker que reservou a linha.
dispatched_at	TIMESTAMPTZ	TEXT	sim	Entrega confirmada.
last_error	TEXT	TEXT	sim	Última falha (até 2.000 caracteres).

Índices parciais: `ix_outbox_pending (next_attempt_at, id) WHERE dispatched_at IS NULL` e `ix_outbox_dispatched (dispatched_at) WHERE dispatched_at IS NOT NULL` — a fila ativa fica pequena mesmo com milhões de linhas entregues.

saga_instances

Coluna	PostgreSQL	SQLite	Nulo	Descrição
saga_id	TEXT PK	TEXT PK	não	Identificador da instância.
saga_type / correlation_id	TEXT	TEXT	não	Únicos juntos: uma saga por tipo e pedido.
state	TEXT	TEXT	não	Estado atual (AwaitingPayment, Compensating...).
data	JSONB	TEXT	não	Dados do passo (ex.: referência do pagamento).
version	BIGINT	INTEGER	não	Incrementada a cada transição.
created_at / updated_at	TIMESTAMPTZ	TEXT	não	Ciclo de vida; índice em `updated_at DESC`.

Copyright (©) 2026 Clara Portes

processed_messages

Coluna	PostgreSQL	SQLite	Nulo	Descrição
consumer + message_id	TEXT PK composta	TEXT PK composta	não	Mensagem já tratada por um consumidor (inbox).
processed_at	TIMESTAMPTZ	TEXT	não	Momento do processamento.

06 Lado de escrita: broker

Persistência opcional do broker para sobreviver a reinícios (Broker:PersistMessages).

broker_messages

Coluna	PostgreSQL	SQLite	Nulo	Descrição
topic + msg_offset	TEXT + BIGINT PK	TEXT + INTEGER PK	não	Mensagem por tópico e offset.
payload	BYTEA	BLOB	não	Bytes publicados, sem interpretação.
stored_at	TIMESTAMPTZ	TEXT	não	Momento do append no broker.

Gravada em lotes de até 512 mensagens por transação a partir de um `Channel` (group commit), com `ON CONFLICT DO NOTHING`. A cada 500 lotes, mensagens fora da janela `RetainedMessagesPerTopic` são removidas.

consumer_offsets

Coluna	PostgreSQL	SQLite	Nulo	Descrição
topic + group_name	TEXT PK composta	TEXT PK composta	não	Grupo de consumo.
committed_offset	BIGINT	INTEGER	não	Menor offset ainda não confirmado.
updated_at	TIMESTAMPTZ	TEXT	não	Última gravação (a cada 5 s e no desligamento).

> **Recuperação.** Na inicialização o broker relê o final de cada tópico e os offsets; grupos voltam a entregar a partir do último offset confirmado. Mensagens confirmadas após a última gravação podem ser reentregues — consumidores são idempotentes por contrato.

07 Lado de leitura

Visões desnormalizadas mantidas pelas projeções, consultadas com AsNoTracking.

Copyright (©) 2026 Clara Portes

Tabela	Chave	Colunas principais	Índices
rm_orders	order_id	customer_ref, currency, status, item_count, total_cents, payment_ref, tracking_code, failure_reason, version, created_at, updated_at	(status, updated_at), updated_at
rm_order_lines	id	order_id, sku, quantity, unit_price_cents	order_id
rm_daily_sales	day (DateOnly)	paid_orders, revenue_cents, refunds, refunded_cents	—
rm_checkpoints	projection	position, updated_at	—

Regras do modelo de leitura

- **Uma consulta, uma tabela.** Cada tela deve ser atendida sem joins custosos; se precisar, crie uma nova visão e uma nova projeção.
- **Checkpoint junto com os dados.** Visões e `rm_checkpoints.position` são salvos na mesma transação.
- **Idempotência por versão.** A projeção ignora eventos com `stream_version` menor ou igual ao `version` da linha.
- **Descartável.** Mudou o formato? Apague e reconstrua por replay (`POST /api/projections/rebuild`), sem migração de dados.
- **Sem escrita externa.** Somente as projeções escrevem nas tabelas `rm_`.

Copyright (©) 2026 Clara Portes

08 Transações e concorrência

Cada operação de escrita tem um mecanismo explícito contra condições de corrida.

Append de eventos

```
BEGIN;
SELECT pg_advisory_xact_lock(74201);
SELECT version FROM streams WHERE stream_id = @id FOR UPDATE;
INSERT INTO streams (...) VALUES (...)
  ON CONFLICT (stream_id) DO UPDATE SET version = excluded.version, updated_at =
excluded.updated_at;
INSERT INTO events  (...) VALUES (...);
INSERT INTO outbox  (...) VALUES (...);
COMMIT;
```

- **Passo a passo:** no SQLite a transação abre com `BEGIN IMMEDIATE` e não há advisory lock nem `FOR UPDATE`; se a versão lida diferir da esperada, a transação é abortada antes de qualquer escrita; `events` e `outbox` recebem uma linha por evento.
- **Controle otimista:** o comando informa a versão que leu; divergência aborta. O dispatcher de comandos repete até 3 vezes recarregando o agregado.
- **Última barreira:** `uq_events_stream_version` rejeita duas gravações da mesma versão mesmo se a checagem for contornada.
- **Ordem global:** a advisory lock serializa os appends no PostgreSQL para que `global_position` siga a ordem de commit — sem isso, uma projeção poderia avançar o checkpoint além de uma transação ainda aberta e pular eventos.

Reserva do outbox

```
UPDATE outbox SET locked_until = @agora + @lease, locked_by = @worker
WHERE id IN (SELECT id FROM outbox
             WHERE dispatched_at IS NULL AND next_attempt_at <= @agora
               AND (locked_until IS NULL OR locked_until < @agora)
             ORDER BY id LIMIT @lote FOR UPDATE SKIP LOCKED)
RETURNING id, message_id, topic, payload, headers, attempts, created_at;
```

Projeções e sagas

Projeções rodam com um semáforo por processo e checkpoint transacional. Sagas usam `uq_saga_correlation` e upsert com incremento de versão; a inbox impede reprocessar a mesma mensagem.

Copyright (©) 2026 Clara Portes

09 Imutabilidade e correções

O histórico é auditável porque não pode ser reescrito.

```
CREATE OR REPLACE FUNCTION brado_events_are_immutable() RETURNS trigger AS $$
BEGIN
    RAISE EXCEPTION 'events é append-only: UPDATE/DELETE não são permitidos';
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_events_immutable BEFORE UPDATE OR DELETE ON events
    FOR EACH ROW EXECUTE FUNCTION brado_events_are_immutable();
```

No SQLite, dois triggers (`trg_events_no_update` e `trg_events_no_delete`) fazem `RAISE(ABORT, ...)`.

Como corrigir dados

Situação	Abordagem correta
Valor errado registrado	Novo evento de correção (ex.: `OrderPriceCorrected`) aplicado pelo agregado e pelas projeções.
Operação que não deveria ter ocorrido	Evento compensatório (como `PaymentRefunded` + `OrderCancelled`).
Visão de leitura inconsistente	Rebuild das projeções — nunca editar `rm_` à mão.
Evento com formato antigo	Upcaster na leitura (ver Evolução futura), sem reescrever a linha.

> **Acesso administrativo.** Em produção, o usuário da aplicação deve ter apenas `INSERT` e `SELECT` em `events` ; o trigger é a segunda linha de defesa.

Copyright (©) 2026 Clara Portes

10 Retenção e manutenção

O que cresce, o que é limpo automaticamente e o que exige rotina operacional.

Objeto	Crescimento	Manutenção automática	Rotina recomendada
events	Ilimitado (é o histórico)	Nenhuma	Particionar e arquivar faixas antigas (seção 11).
snapshots	1 linha por stream	Sobrescrito a cada N eventos	Nenhuma.
outbox	1 linha por evento	Entregues > 7 dias removidos a cada hora	Alertar se pendentes crescerem.
processed_messages	1 linha por mensagem e consumidor	Nenhuma	Remover linhas com mais de 30 dias (janela de reentrega).
broker_messages	Até a retenção por tópico	Trim a cada 500 lotes	Ajustar `RetainedMessagesPerTopic`.
rm_*	Proporcional aos agregados	Projeções	Rebuild quando o formato mudar.

PostgreSQL

- Autovacuum mais agressivo em `outbox` e `processed_messages` (muitas atualizações e remoções): `autovacuum_vacuum_scale_factor = 0.02`.
- `events` recebe apenas inserções: o custo de vacuum é baixo; mantenha `fillfactor` 100.
- Backup contínuo (WAL archiving / PITR) do banco de escrita. O de leitura dispensa backup.

SQLite

- WAL ligado, `synchronous = NORMAL`, `busy_timeout = 10000` e transações `BEGIN IMMEDIATE`.
- Adequado para desenvolvimento, demonstrações e instâncias únicas; para vários escritores, use PostgreSQL.

Copyright (©) 2026 Clara Portes

11 Evolução futura

Diretrizes que o banco deve seguir conforme o volume e os requisitos crescerem.

---

1. Particionamento de events

Ao passar de dezenas de milhões de linhas, converter `events` em tabela particionada por faixa de `global_position` (ou por mês de `recorded_at`). Restrições únicas passam a incluir a chave de partição; a unicidade global de `event_id` migra para uma tabela de índice `event_ids(event_id PK, global_position)`.

```
CREATE TABLE events (...) PARTITION BY RANGE (global_position);
CREATE TABLE events_p000 PARTITION OF events FOR VALUES FROM (1) TO (50000001);
CREATE INDEX ix_events_recorded_brin ON events USING BRIN (recorded_at);
```

2. Versionamento do schema dos eventos

- Todo evento já grava `schemaVersion` em `metadata`. Mudanças incompatíveis criam a versão seguinte.
- Leitura com **upcasters** encadeados (v1 → v2 → v3) no `EventTypeRegistry`; o banco nunca é reescrito.
- Renomear evento = registrar alias do nome antigo, nunca alterar `event_type` gravado.

3. Migrações versionadas

- Escrita: scripts numerados (`V001__base.sql`, `V002__particionamento.sql`) com tabela `schema_history`, executados antes dos workers.
- Leitura: migrações do EF Core no lugar da criação automática quando houver mais de uma instância.

4. Escala de leitura e notificação

- Réplicas de leitura para as visões `rm_` e para replays pesados.
- `LISTEN/NOTIFY` no PostgreSQL para acordar projeções e outbox entre servidores, mantendo o polling como reserva.
- Projeções novas em bancos especializados (busca, séries temporais) alimentadas pelo mesmo log.

5. Arquivamento

Faixas antigas de `events` vão para armazenamento frio (Parquet) com hash encadeado por lote; snapshots recentes garantem que nenhum agregado ativo dependa do arquivo.

6. Dados pessoais (LGPD)

- Eventos não devem carregar dados pessoais em claro; usar referências (como `customerRef`) ou campos cifrados.
- **Crypto-shredding:** chave por titular em tabela separada (`subject_keys`); apagar a chave torna os dados ilegíveis sem violar a imutabilidade do log.

7. Multi-tenant e segurança

- Coluna `tenant_id` em todas as tabelas de escrita, compondo chaves e índices, com *Row Level Security*.
- Papéis distintos: aplicação (INSERT/SELECT em `events`), workers (UPDATE no `outbox`), leitura (somente `rm_`); tipos nativos `uuid` quando o SQLite deixar de ser alvo.

Copyright (©) 2026 Clara Portes

12 Checklist e DDL de referência

Verificações antes de colocar um ambiente no ar e o núcleo do script do PostgreSQL.

#	Verificação
1	Event store e modelo de leitura em bancos separados (ou ao menos schemas distintos).
2	Usuário da aplicação sem permissão de UPDATE/DELETE em `events`.
3	Backup contínuo (PITR) do banco de escrita testado com restauração.
4	Alertas para outbox pendente, lag dos grupos e falhas de projeção.
5	Rotina de limpeza de `processed_messages`.
6	Fuso de servidor indiferente: a aplicação grava sempre UTC.
7	Plano de particionamento definido antes de 50 milhões de eventos.

```
CREATE TABLE IF NOT EXISTS streams (
    stream_id TEXT PRIMARY KEY, stream_type TEXT NOT NULL,
    version BIGINT NOT NULL CHECK (version >= 0),
    created_at TIMESTAMPTZ NOT NULL, updated_at TIMESTAMPTZ NOT NULL);

CREATE TABLE IF NOT EXISTS events (
    global_position BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    stream_id TEXT NOT NULL REFERENCES streams (stream_id),
    stream_version BIGINT NOT NULL CHECK (stream_version > 0),
    event_id TEXT NOT NULL, event_type TEXT NOT NULL,
    payload JSONB NOT NULL, metadata JSONB NOT NULL,
    occurred_at TIMESTAMPTZ NOT NULL, recorded_at TIMESTAMPTZ NOT NULL,
    CONSTRAINT uq_events_stream_version UNIQUE (stream_id, stream_version),
    CONSTRAINT uq_events_event_id UNIQUE (event_id));

CREATE TABLE IF NOT EXISTS outbox (
    id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    message_id TEXT NOT NULL UNIQUE, topic TEXT NOT NULL,
    payload JSONB NOT NULL, headers JSONB NOT NULL,
    created_at TIMESTAMPTZ NOT NULL, attempts BIGINT NOT NULL DEFAULT 0,
    next_attempt_at TIMESTAMPTZ NOT NULL, locked_until TIMESTAMPTZ NULL,
    locked_by TEXT NULL, dispatched_at TIMESTAMPTZ NULL, last_error TEXT NULL);
CREATE INDEX IF NOT EXISTS ix_outbox_pending ON outbox (next_attempt_at, id) WHERE dispatched_at IS
NULL;
```

O script completo (snapshots, sagas, inbox, journal, offsets e triggers) está em `src/Brado.EventStore/Dialects/PostgresDialect.cs` e `SqliteDialect.cs`.

Copyright (©) 2026 Clara Portes